# Proof Engine Infrastructure: A Fail-Closed Claim-Graph Method for Accountable AI-Assisted Mathematical Research

Alex Chengyu Li[1*]

[1*]Independent Researcher, Tokyo, Japan, ORCID: 0009-0008-4516-8946.

Corresponding author(s). E-mail(s): cl7076@nyu.edu;

**Abstract**

**Background:** AI systems can produce proof sketches, formal code, solver artifacts, and candidate strategies faster than research communities can assess them. Long-running and parallel agents can also hallucinate, lose problem context, duplicate obligations, or return incompatible formulations. Disclosure and local checking alone do not show which claim has been earned, whether its dependency route is closed, or whether evidence remains current after correction.

**Methods:** Proof Engine Infrastructure was developed as a fail-closed method for claim-level research reporting. An agent-to-claim control plane externalizes claims, obligations, and receipts in a typed directed hypergraph. Its minimum loop declares the target and admissible roots, decomposes claim paths, dispatches work, assigns edge-adequate evidence boundaries, binds checked inputs and outputs, composes earned edges, and recomputes the frontier. A contrasting-case analysis examined three completed projects with different mathematical objects and two closure architectures. An odd-sum case visualizes parallel checks, asynchronous returns, frontier selection, an unresolved continuation, and retained knowledge.

**Results:** A new uniform Hamilton classification tested paper-to-kernel binding. A new exact all-N solution of Erdős Problem 848 completed the finite range left by earlier GPT-5-assisted work, using compression, certificates, semantic checking, complete coverage, and kernel replay. A rational-Dyck-path project repeated direct closure on a new object. The running graph showed five parallel checks, one selected open frontier, and retained partial and negative routes without claiming closure. A separate large ongoing proof program provided qualitative

operational evidence for concurrent proof search, formalization, review, correction, and assembly. Wider parallelism was useful but token- and coordination-intensive. **Conclusions:** Fail-closed agent-to-claim graphs provide a correction-aware control layer between AI-assisted generation, heterogeneous verification, and publication. Three completed projects provide bounded transfer evidence within mathematics, while ongoing use supports practical concurrent development. The next research stage is a graph-native Proof Engine 2.0 protocol for typed handoffs, receipt-only trust, conflict resolution, dependency-aware scheduling, and resource accounting, followed by matched-task quantitative evaluation.

**Keywords:** AI-assisted research, research integrity, claim graphs, agent-to-claim control, research reporting, formal verification, proof engineering

# 1 Background

AI-assisted mathematics is often framed as a question about model capability: can a system prove a theorem, solve an olympiad problem, or translate an informal argument into a proof assistant? These are important generation questions. They do not finish the work of research. Once a plausible argument, formal file, certificate, or computation exists, a project must determine what that object actually establishes and how far its support may travel.

That post-generation task has five connected parts. First, the project must identify the exact mathematical object being checked. Second, it must decide whether the available check is adequate for the claim. Third, it must determine whether the checked local pieces compose into the announced target. Fourth, it must preserve or withdraw their status when definitions and dependencies change. Finally, it must keep the next unresolved obligation visible while several people or agents work at once. These are not separate administrative concerns. Together they determine the current research state.

A simple endpoint mismatch shows why local success is insufficient. Suppose a formal file compiles, but a later reader discovers that its theorem uses a different endpoint convention from the paper. The kernel result remains valid for the formal statement it checked. The paper claim nevertheless loses its support, because the

checked statement is not the statement consumed by the argument. Rebuilding the same file or refreshing its hash cannot repair that semantic break. Downstream conclusions must be reconsidered until the binding is corrected and checked again.

Long-running and parallel agents create the same research-state problem at a larger scale. A worker may forget a load-bearing constraint, hallucinate a bridge, or return a proposition at the wrong scope. Several workers may duplicate one obligation, use incompatible definitions, or advance branches that no longer serve the active target. The difficulty is not merely that agents can make mistakes. It is that accepted facts, tentative ideas, failed routes, definitions, and unresolved dependencies cannot remain only in transient prompt histories if their outputs are expected to compose.

Existing approaches cover important parts of this chain. Model and benchmark evaluations measure proof search, autoformalization, or task performance [36, 37, 39–41]. Agentic provers add planning, retrieval, parallel workers, repair, and formal feedback [25–27]. Proof assistants answer a narrower question: does this formal artifact prove this formal statement? Research-integrity proposals address another part. They use structured disclosure and workflow documentation to make AI provenance and human responsibility more visible [1, 2].

Each contribution is useful. None of them, by itself, records whether the checked object matches the paper claim. None determines whether local results close the full dependency route. None says which conclusions must be withdrawn after a correction. These are reproducibility questions in the broad sense of keeping the scholarly record accurate, inspectable, and self-correcting [3].

Proof Engine Infrastructure addresses this missing layer by representing the research state explicitly. The exact target, its definitions, claim paths, work units, evidence receipts, and active frontier are stored outside any individual agent context. Generated outputs begin as candidates. A candidate can carry an inference only after an adequate check is bound to the exact proposition and current semantic object. Typed graph

edges then record how earned claims compose; failed and superseded routes remain auditable without contributing to closure. When a load-bearing object changes, affected edges lose current reachability and must be earned again; a name or refreshed hash carries no inherited status.

The persistent unit is therefore the claim. Workers are transient. Proof Engine uses an *agent-to-claim control plane*: human and machine agents receive typed claim obligations, while claims, contracts, receipts, and frontier state remain available after those agents exit. It differs from a graph of agent communication or authority. Agents may generate and exchange artifacts, but only a bound receipt may change a claim's inferential status.

Under its stated contracts and promotion gates, this architecture joins correctness and coordination without confusing them. Independent obligations can be developed concurrently because their required inputs and outputs are typed in advance. Asynchronous returns can arrive in any order, but trust still enters through the same checking, binding, and composition rules. The graph cannot prevent hallucination or context loss inside a model. It keeps an unbound return from changing the mathematical status of the target. The same protection applies to stale evidence and duplicated work. An incorrectly specified contract or mistaken binding remains possible; the limitations below preserve that human-review boundary.

The method was shaped by two public projects containing new mathematics. It was then reused successfully on a third object. The first project gives a uniform Hamilton cycle and path classification. It tests whether a human argument and premise-free Lean endpoints express the same result [15]. The second project gives an exact all-$N$ solution of Erdős Problem 848 [17]. It tests whether structural mathematics, exhaustive certificates, semantic checking, and kernel replay compose into one closed theorem. The rational-Dyck-path project reuses the direct mathematical-to-kernel pattern for a

counterexample and cover results [21]. A companion paper supplies the local verification-profile concept [16]. This article develops the global state and transition method that composes such records over time.

## 1.1 Contributions

1. An agent-to-claim control architecture. It makes claims and receipts persistent while treating agents as replaceable workers. It decomposes an exact consumer into typed claim paths that may be developed concurrently. Within a fixed root snapshot, only evidence-bearing earned edges may change closure (Sections 2.3–2.7).
2. A temporal semantics. Immutable definition contracts identify the mathematical subject. An admissible root registry prevents closure by declaration. Invalidation and re-earning let later corrections withdraw inferential credit without erasing history (Sections 2.5 and 2.8).
3. A research-control rule. The active frontier remains tied to the exact consumer. Formal, certificate, witness, and replay boundaries are chosen edge by edge using adequacy and total verification cost (Sections 2.10–2.12).
4. A contrasting-case comparison. It covers direct kernel closure, certificate-heavy exact closure, and reuse on a third mathematical object. A running claim graph shows earned, open, partial, and refuted routes. A separate stress case examines concurrent development (Section 3).

## 1.2 Scope and non-goals

The subject of the article is research state: how generated material acquires, loses, and regains inferential force. SAT solving, proof-assistant design, and certificate formats enter only as possible checking boundaries. The article therefore publishes a method comprising evidence identity, promotion and correction rules, frontier discipline, and closure-respecting composition. Software distribution and implementation tutorials

lie outside its scope. Concrete deployments supply evidence and counterexamples, while their commands, schemas, and project-specific source trees remain outside the exposition. Conversely, merely drawing a graph around agent outputs or attaching checker badges fails to instantiate the method. It must implement the closed-loop claim, binding, promotion, correction, and frontier invariants stated below.

# 2 Methods

## 2.1 Method at a glance

The background described one research-state problem with five failure modes. This section builds the response in dependency order. It starts by freezing the target. It then decomposes that target into claim paths. Only after those steps does it explain how evidence is checked, bound, and composed. Correction and frontier selection come later because they operate on the state created by those earlier steps. Table 1 maps each failure mode to the mechanism that addresses it. The method is a control layer with a bounded function. It limits local acceptance to the object, route, and time for which support has actually been earned.

The response is instantiated by the following minimum closed loop:

1. declare the exact target, seal an admissible root registry, and freeze the load-bearing semantic subject.
2. decompose its consumer into typed claim paths without changing scope.
3. dispatch independent path obligations to human or machine workers, concurrently where their input contracts permit.
4. assign each edge an adequate verification boundary, such as a direct check, formal endpoint, independently replayable certificate, or replicated computation, according to its object and role.

| Operational pain point | Why local checking is insufficient | Method response |
|---|---|---|
| The wrong object passes | A checker can accept a changed endpoint, normalization, carrier, or encoding | Reference grounding, definition contracts, and exact claim binding |
| Correct pieces fail to prove the target | Local lemmas may leave a case, assumption, overlap, or dependency route open | A typed claim hypergraph, all-or-nothing promotion, and derived closure |
| Earlier evidence becomes stale | A correction can change the subject while an old receipt still appears green | Immutable supersession, invalidation, and fresh re-earning |
| Agent context and concurrency fragment the state | A worker may forget a constraint or invent a bridge; parallel workers may duplicate an obligation or return incompatible objects | Externalized graph state, typed work units, dependency-aware dispatch, and receipt-bound asynchronous integration |
| Activity displaces the real bottleneck | A shorter or more formal route may solve a stronger, optional, or different problem | Exact-consumer frontier classification and graph-level verification cost |

**Table 1** Five manifestations of one research-state problem and the corresponding method components.

5. bind every accepted artifact to its declared inputs, exact output, and downstream consumer.
6. promote, retire, or invalidate edges, compose only earned routes, and recompute the active frontier.
7. repeat after each material return or correction, exposing a publication projection only at the closure actually earned.

The first five steps establish what a result means and what would count as checking it. The last two decide whether the checked pieces may change the target's status. They also identify the work that remains. This division is useful when reading the more formal definitions below.

**Principle 1** (Minimal closed-loop instantiation)**.** *A research workflow instantiates Proof Engine Infrastructure only when its core operations are connected in one auditable*

*loop. Those operations are candidate generation, admissible-root control, claim-path decomposition, edge-adequate checking, exact input-output binding, all-or-nothing promotion, derived closure, correction, and frontier recalculation. Parallel agents, a graph database, or checker labels alone are insufficient.*

**Principle 2** (Context-externalized coordination)**.** *The load-bearing state must exist outside any individual agent context. It includes the target, admissible roots, definition contracts, dependencies, work-unit state, evidence receipts, and active frontier. A returned artifact remains a candidate until it satisfies its contracted output and promotion boundary. Repeated returns to the same edge do not create additional inferential force.*

The four functional roles (generation, verification, composition, and publication) occur inside this loop. The rest of the Methods section follows the same order. It first separates generation from verification. It then defines positive evidence and explains how local results compose. Correction comes next. Frontier choice and verification cost come last.

One running example accompanies that progression. Let $S_n = \sum_{k=0}^{n-1}(2k+1)$, with declared target $T : \forall n \geq 0,\ S_n = n^2$. Three initially open routes attempt the target: induction, symmetric pairing, and an equal-term shortcut. The induction route then decomposes into independently checkable base, recurrence, and algebra obligations. The example is deliberately elementary so that the reader can concentrate on graph behavior. The example will be revisited several times. Each return answers one concrete question: what was generated, what was checked, what status changed, and what remains open.

## 2.2 Design and evidence base

The method was developed iteratively against completed work in one AI-assisted mathematical research program. The cases share that provenance, but they do not ask the

same mathematical question. One is a graph classification. One is an extremal number-theory theorem solved for every $N$ through structural mathematics and exhaustive certificates. The third concerns order questions for rational Dyck paths.

The cases were selected to maximize contrast in mathematical object and evidentiary architecture. They were not selected to estimate a population effect. Each included case needed a stable record, a substantive machine-checking boundary, and a status transition that tested part of the method. Two main regimes met those conditions: direct constructive kernel closure and certificate-heavy exact closure. The rational-Dyck-path project then tested the reuse of direct closure on a third object.

The unit of analysis was a claim edge and its status transition. The truth of one theorem was not used to validate another. The evidence base consisted of public mathematical manuscripts and formal packages for the successful projects. For each case, the analysis recorded the intended subject, checking boundary, downstream consumer, dependency route, correction behavior, and final publication label. The comparison is qualitative and descriptive; no statistical analysis was used. Additional file 1 provides the reporting checklist for these fields without exposing project-specific commands, source trees, or private implementation details.

Separately, one large ongoing proof program served as a development stress case for concurrency. It exercised parallel claim-path work in mathematical search, formalization, independent review, correction, and final assembly. Both narrower and wider parallel configurations were used. The operational record contains sustained agent-scoped milestones, typed routes, checks, and attempt histories. Wide configurations consumed substantial model tokens and coordination effort, but were useful when work units were independent and returned reusable receipts. The target remains open, and its research materials are not released. The program is therefore reported as method-development stress evidence. It is not one of the three completed mathematical deployments. It

supports operational feasibility only at a qualitative workflow level and motivates the protocol and matched-task evaluation developed as future work below.

## 2.3 From generated artifact to reportable research claim

The loop begins with a candidate. An AI agent may propose a proof strategy or write an encoder. It may also produce a proof sketch, translate a definition, search examples, or summarize failed attempts. All of these activities belong to generation. Their outputs acquire positive inferential force only after an appropriate verification boundary has been crossed.

**Claim 1** (Evidentiary status is output-relative)**.** *An output may carry inferential force only through an independent verification procedure with stated scope. The identity of the human or machine that proposed it supplies provenance, not inferential force.*

The design objective is to support multiple fallible generation processes while making their mathematical outputs inexpensive to verify independently.

The claim graph is therefore a shared control plane for concurrent research workers. A single-agent transcript is a different object. A route is first decomposed at the claim level. Separate agents may then develop its producers or attack an independent route. They may instead search for a counterexample or construct a checker. Their private search trajectories may differ. Their returns still meet the same typed consumer contracts.

**Definition 1** (Claim-path work unit)**.** *A claim-path work unit names a parent consumer and the exact child proposition to be produced. It also records the input contract, definition contract, admissible verification boundaries, and evidence fields required for promotion. A worker may choose how to generate the candidate. It may not silently redefine the contracted output.*

**Definition 2** (Agent-to-claim control plane)**.** *An agent-to-claim control plane maps transient worker actions to persistent claim state. Agents receive claim-path work units and may return artifacts, counterexamples, or proposed transports. No worker identity, message, or handoff changes reachability. A state transition occurs only when the returned object is checked, bound to its contracted claim, and admitted under the active snapshot.*

When these contracts and gates are enforced, this separation makes concurrency safe at the research-state level. Agents can work on disjoint or competing paths without sharing epistemic authority, and asynchronous returns can be evaluated in any order against the active snapshot. The graph accepts a return through its evidence receipt and binding, not through the identity, confidence, or persistence of the worker that produced it.

Externalized state also changes failure handling. If a worker loses context, the contracted proposition and definitions remain fixed outside that context. If it hallucinates a lemma, the unbound return stays candidate. If two workers produce the same result, both artifacts may remain auditable, but the shared edge is earned only once. If their scopes differ, the graph records distinct claims instead of merging them through prose.

The same state can guide allocation. A dispatcher can avoid knowingly repeating an already earned or retired work unit. It can run independent obligations in parallel. It can delay a dependent unit until its inputs exist, and it can reuse a current receipt across several consumers. These mechanisms supported useful concurrent progress in ongoing development. They also define variables for a controlled comparison. The existing record was not collected under a fixed latency, token, or throughput protocol and cannot establish net acceleration.

In the running example, induction, pairing, and the shortcut are therefore recorded first as generated routes instead of three competing degrees of proof. The engine may dispatch the induction base, recurrence identity, algebraic closure, symmetric pairing,

and shortcut counterexample as five separate work units. Their checks can run in parallel because the required outputs are typed in advance. Before their bound receipts return and compose, none changes the status of $T$.

The work-unit mechanism separates responsibilities that are often collapsed in one description of an AI system. Producing a candidate is one function. Checking it is another. Composition and publication are different again. The same person or program may perform several functions, but success in one supplies no automatic authority in the next.

**Definition 3** (Functional role)**.** *A functional role is a contribution to the formation or assessment of a mathematical claim. Roles describe the activity (generation, verification, composition, or publication). The identity of the performer is recorded separately. Trust follows the relevant evidence boundary and remains independent of the role label.*

Table 2 gives a minimal separation of functions. A human or machine participant may perform more than one function, but performing one function confers no epistemic authority associated with another.

| Function | Contribution | Remaining boundary |
|---|---|---|
| Generation | Candidate claims, strategies, examples, or proof objects | Correctness remains open |
| Verification | A check of an artifact under a stated verification boundary | Downstream sufficiency still requires claim binding and composition |
| Composition | An inference from verified components to a larger claim | Component adequacy must already be established |
| Publication | A stable, inspectable presentation of claims and evidence | Public wording is limited to the status already earned |

**Table 2** Functional separation prevents provenance or authorship from substituting for evidence.

### 2.3.1 The generation-verification interface

**Definition 4** (Generation-verification interface)**.** *A generation-verification interface is the part of a research infrastructure that exposes the evidentiary record of each substantive claim to the larger claim graph.*

The companion paper calls the corresponding local object a *verification profile* [16]. A profile records a claim and the object that represents it. It also records the checking procedure, the scope of the check, the remaining assumptions, and a stable identifier. This paper leaves that claim-level taxonomy to the companion and asks how many such records can be composed into a research state whose closure is derivable.

The interface forms a boundary between local evidence and global inference. It can accept different kinds of verification profile without treating them as equivalent. For each profile, it exposes the exact proposition that has been licensed. A local status can then enter the graph without silently becoming a stronger claim.

## 2.4 What counts as positive evidence

Separating generation from verification solves a provenance problem, but it leaves a semantic one. A checker can be perfectly correct about the bytes it receives while those bytes encode the wrong endpoint, normalization, or theorem. The response is a three-part positive-evidence obligation: identify the intended object, check an artifact, and bind the two at the exact claimed scope.

A verification profile records how an artifact was checked. Any value, structure, or theorem may later contribute evidence to the larger graph. Before that happens, the graph asks for more than a successful check. The artifact must be grounded in the intended research object. It must also be bound to the exact claim whose status would change.

**Definition 5** (Reference grounding)**.** *Reference grounding for a claim $C$ is a versioned specification of the object to which $C$ refers. It records the governing definitions and normalization. It records where input or reference data came from and which dependencies are in use. It also records the baseline cases or calibration facts that identify the intended representation.*

Reference grounding is domain dependent. For a finite mathematical value it may contain the instance family, input data, known cases, and encoding convention. For a formal theorem it may contain the exact definitions, imported theory, and the correspondence between the paper statement and the formal statement. For an empirical object it may contain raw observations, calibration data, and the measurement protocol. Each object fixes what is being checked, and the three object classes remain non-interchangeable.

**Definition 6** (Machine-checkable evidence)**.** *Machine-checkable evidence for $C$ is a versioned artifact, an independently executable checking procedure, and an acceptance record produced under a declared trust and dependency policy.*

The machine-checkable artifact may be a proof term, certificate, witness, exhaustive computation, symbolic derivation, or independently rerunnable analysis. No single proof assistant, solver, or certificate language is required. The selected mechanism must expose an acceptance relation whose scope is adequate for $C$.

**Definition 7** (Claim binding)**.** *A claim binding is an inspectable bridge showing that the reference-grounded object and the machine-checked object express the exact claim $C$. Their domain, definitions, normalization, endpoints, and declared assumptions must agree.*

Let $G(C)$, $M(C)$, and $B(C)$ denote reference grounding, machine acceptance, and exact claim binding. The positive-evidence obligation is

$$\text{PositiveEvidence}(C) \quad \Longrightarrow \quad G(C) \wedge M(C) \wedge B(C).$$

Each conjunct blocks a different failure. Reference data identifies the object but supplies no new theorem. A machine proof can certify the wrong encoding when reference grounding is absent. A semantic bridge establishes correspondence, yet still needs an accepted artifact on the machine side. Positive evidence requires the three boundaries together.

In plain terms, the three letters ask three questions. Is this the intended object? Did the declared checker accept an artifact? Does that accepted artifact support the exact sentence the project wants to use? If any answer is missing, the evidence remains local and candidate.

**Principle 3** (Technology-independent positive evidence)**.** *Every non-root positive claim whose promotion changes reachability within a fixed root snapshot must be supported by reference grounding, machine-checkable evidence, and an exact claim binding. The technology realizing the machine check may be replaced. The replacement must verify the same claim at no weaker scope, and it must expose its trust assumptions.*

**Definition 8** (Equivalent checking realizations)**.** *Two machine-checking realizations are equivalent for a claim $C$ when both use the same reference grounding and support the same proposition. Their scope and dependency policy must also agree. In each case, the evidence must be independently replayable. A stronger result is not equivalent merely because it is stronger. It may replace the earlier realization only when it is earned under compatible grounding and trust policy, introduces no stronger assumptions or weaker scope, and carries an earned transport to $C$.*

The infrastructure can evolve without evidentiary drift. A proof-assistant kernel may replace a certificate checker. A certified checker may replace raw solver replay. An independently implemented verifier may replace an earlier script. The graph need not privilege the original technology. It must preserve the proposition established, the reference object, and every remaining assumption.

## 2.5 Keeping the mathematical object fixed

The positive-evidence rule is still incomplete if the identity of the checked claim can drift over time. A path and a cryptographic digest identify bytes; they do not establish that those bytes still describe the same mathematical object. A refreshed hash can faithfully authenticate a corrupted formula, a changed endpoint, or a replacement theorem that no longer implies the claim consumed downstream.

**Definition 9** (Definition contract)**.** *A definition contract for a load-bearing mathematical object records the symbol, namespace, and literal meaning. It records the domain, codomain, quantifier order, ranges, and endpoints. It also fixes normalization, sign or orientation, occurrence multiplicity, and the transitive definitions that determine how the object is used in the graph.*

Not every project needs every field. The requirement is that every mathematically material degree of freedom be fixed somewhere inspectable. For a graph theorem, adjacency and endpoint conventions may be load-bearing. For an analytic estimate, normalizer, weight, orientation, parameter role, and error sign may be load-bearing. For a certificate, the encoder, instance, decoder, and acceptance predicate may be load-bearing. Treating all three as one passing file loses the object on which the evidence depends.

**Definition 10** (Evidence identity)**.** *The identity of an evidence receipt $r$ for an edge $e$ is*

$$\mathrm{Id}(r) = \big(e, \Sigma_e, \Delta_e, A_e, K_e, D_e, P_e\big),$$

*The pair $(\Sigma_e, \Delta_e)$ records the semantic subject and its transitive definition-contract closure. The remaining fields record the checked artifacts $A_e$, checker and acceptance predicate $K_e$, dependency closure $D_e$, and trust and promotion policy $P_e$.*

This identity is intentionally stronger than an artifact hash. If an artifact changes while the semantic subject remains fixed, the new bytes require a fresh check. If the semantic subject changes, the correction requires a new definition identity or an explicit supersession relation even when the source path is unchanged. If policy changes, an older receipt remains historical evidence under its recorded policy but cannot silently count as current acceptance under the new one.

**Principle 4** (Immutable semantic correction)**.** *A sealed definition contract is append-only. A correction creates a new definition identity and an explicit supersession or transport relation; it does not mutate the meaning under the old identifier.*

Append-only correction serves two purposes. First, an old receipt remains interpretable: it continues to say exactly which object was checked. Second, current routes can be required to consume the corrected definition and can be forbidden from reintroducing a retired object through another apparently complete dependency chain. This is stronger than adding a warning to prose.

Source integrity is part of the same problem. A research system may use exact anchors, required semantic tokens, encoding checks, or independent parses to detect damaged mathematical text before accepting a refreshed identity. These are realization choices. The methodological invariant is that a content refresh cannot legitimize a source whose load-bearing semantics have already changed without review.

**Proposition 1** (Subject preservation)**.** *Suppose a receipt $r$ earns an edge $e$ for semantic subject $\Sigma_e$. After a load-bearing artifact, checker, dependency, or definition update, the identity of $r$ remains historical and cannot license a changed subject. The edge can regain current inferential force in only two ways. A new accepted receipt may establish the same subject under no weaker scope. Alternatively, a new accepted receipt may establish a successor subject and an earned transport may show that it implies the claim consumed by every parent edge.*

*Proof* The earned edge is licensed by the conjunction of reference grounding, machine acceptance, exact claim binding, strength preservation, completeness, and dependency preservation. Changing a load-bearing element removes at least the corresponding conjunct from the current evidence identity. A fresh receipt for the same subject restores the conjuncts directly. For a successor subject, a fresh receipt establishes the new output and an earned implication transport shows that this output is sufficient for the old consumer. Without either route, exact binding or strength preservation is absent, so promotion is inadmissible. □

The running example makes the binding issue literal. Pairing establishes the typed claim $T_{\text{even}} : \forall n \geq 0,\ 2 \mid n \Rightarrow S_n = n^2$. That claim is true and may be earned, but it is not the universal consumer $T$. Changing the quantifier scope in prose would change the mathematical object and would fail to compose evidence for $T$.

## 2.6 Composing local results

Exact local evidence creates the next problem: a target rarely consists of one artifact. It may require several premises at once. It may admit alternative proofs. It may also depend on partial results whose scopes must align. The generation–verification interface describes individual artifacts. A theory of closure must now explain how those artifacts compose.

**Definition 11** (Claim graph)**.** *A claim graph is a directed acyclic hypergraph*

$$\mathcal{G} = (\mathcal{C}, \mathcal{E}),$$

*where each vertex $C \in \mathcal{C}$ is a typed mathematical claim and each hyperedge $e \in \mathcal{E}$ represents one inference. The edge has a finite input set $I_e \subseteq \mathcal{C}$, one output claim $o_e \in \mathcal{C}$, and a declared evidentiary status.*

**Definition 12** (Admissible root registry)**.** *The root registry $R_0$ is a versioned set of claims admitted without a predecessor edge in the current graph. Each root is either an explicit definition or assumption carried into every downstream statement, or an imported claim with a current root receipt that records its exact proposition, source, verification boundary, claim binding, scope, dependencies, and acceptance policy. The declared target, or an equivalent restatement of it, cannot be inserted as a root of an unconditional closure claim. Doing so changes the result to an assumption-relative statement. The registry is fixed within a research snapshot; changing it is a reviewed snapshot transition.*

**Definition 13** (Claim route)**.** *A route to a claim $C$ is a finite derivation sub-hypergraph ending at $C$. Its leaves belong to the admissible root registry, and each non-root claim used by the route is the output of a selected incoming edge whose entire input set is also present in the route. An earned route uses only earned edges.*

Several edges with the same output encode alternative proofs and therefore have OR semantics. A single edge with several inputs encodes an assembly obligation and therefore has AND semantics. Let $R_0$ be the admissible root registry and let $\mathcal{E}_{\text{earned}}$

be the earned edges. Define

$$R_{k+1} = R_k \cup \{\, o_e : \; e \in \mathcal{E}_{\text{earned}},\; I_e \subseteq R_k \,\}, \qquad \text{Reach}(\mathcal{G}) = \bigcup_{k \geq 0} R_k.$$

A target claim $T$ is research-closed relative to the admissible root registry exactly when $T \in \text{Reach}(\mathcal{G})$. The definition is a simple forward process. Start with the admissible roots. Add the output of any earned edge whose inputs are already available, and repeat. The same graph can then show which routes remain open, which obligations are ready, and which inputs are still unavailable.

**Principle 5** (Derived closure)**.** *A non-root claim is closed only when it is reachable from admissible roots through earned edges. Within a fixed snapshot, only a newly earned or invalidated edge can change that reachability. A root change creates a new snapshot and cannot turn the declared target into an unconditional theorem. Confidence, authorship, and the local success of an uncomposed argument leave this relation unchanged.*

### 2.6.1 Edge lifecycle and negative evidence

Local evidence, semantic identity, and graph composition are now defined. The edge states can therefore be introduced before the full promotion formula. A proposed edge begins open. Mathematical scrutiny or an appropriate checker may produce candidate evidence, but the candidate still has no effect on reachability. It becomes earned only after the all-or-nothing gate stated in the next subsection. A refuted or insufficient argument becomes retired. An edge that was previously earned becomes invalidated when its current evidence identity no longer satisfies that gate.

Table 3 distinguishes these edge states and their effects on reachability.

The lifecycle preserves useful failure and correction history without allowing either to alter closure. Retiring one argument leaves its output claim undecided when other incoming edges remain possible; preserving that failed argument gives it no positive

| State | Meaning | Effect on reachability |
|---|---|---|
| Open | An unresolved proof obligation | None |
| Argument-supported candidate | A positive mathematical argument is available at a stated local scope | None |
| Mechanically checked candidate | A stated verification procedure accepted the corresponding artifact | None |
| Earned | The candidate satisfies the admissibility conditions for its output claim | The edge may propagate its output claim |
| Invalidated | Earlier positive evidence no longer matches the current semantic subject, artifacts, dependencies, or policy | None; the old receipt remains auditable but cannot propagate |
| Retired | The argument is invalid, insufficient, or superseded | None; its negative information may be preserved |

**Table 3** Candidate states help organize research while leaving reachability unchanged. Earned edges alone propagate. Publication closure is a separate claim-level projection outside the edge-state taxonomy.

evidentiary force. Invalidating an earned edge retains the earlier acceptance as history while changing whether that receipt licenses the current claim.

### 2.6.2 Running agent-to-claim evolution: the sum of the first odd integers

Consider the familiar target

$$S_n := \sum_{k=0}^{n-1} (2k+1), \qquad T: \ \forall n \geq 0, \ S_n = n^2,$$

with the usual empty-sum convention $S_0 = 0$. Suppose a research session proposes three routes before any route has earned inferential status. Figure 1 records three times and then deliberately stops. At $t_0$, three candidate claim paths enter the graph. At $t_1$, five typed work units are dispatched to concurrent agents. At $t_2$, their asynchronous returns

leave the recurrence identity as the sole active frontier while preserving an earned partial theorem and a retired route. The figure is an operational projection of the claim graph. Its downward arrows mean decomposition, dispatch, and receipt-bound state update. They are not mathematical inference edges. A claim-graph inference would run from earned inputs to an output; no such edge reaches $T$ in the recorded snapshot. The broken continuation and question mark mark research that has not yet happened. They carry no receipt or failed-check status. The frontier might later be proved, checked, and bound; decomposed into child obligations; replaced by a different route; or retired. The figure asserts none of these future transitions, and $T$ therefore remains open.

*Route A: induction.* Its proof edge has three independently checkable inputs. They are the base case $S_0 = 0$, the recurrence $S_{n+1} = S_n + (2n + 1)$, and the algebraic identity $n^2 + 2n + 1 = (n + 1)^2$. The edge can be earned only when all three inputs are current and jointly compose.

*Route B: equal-term shortcut.* Assert the false termwise premise $2k + 1 = n$ for every $0 \leq k < n$, then conclude $S_n = n \cdot n$. The conclusion happens to be the target, but the premise fails already at $n = 2, k = 0$, where $1 \neq 2$. The route must be retired; its correct destination cannot rescue the false premise.

*Route C: symmetric pairing.* For even $n$, pair the first and last summands, then the second and second-last. Each pair sums to $2n$, and $n/2$ pairs give $S_n = n^2$. This earns the restricted claim $T_{\text{even}}$, but its scope alone cannot close $T$ for every $n$.

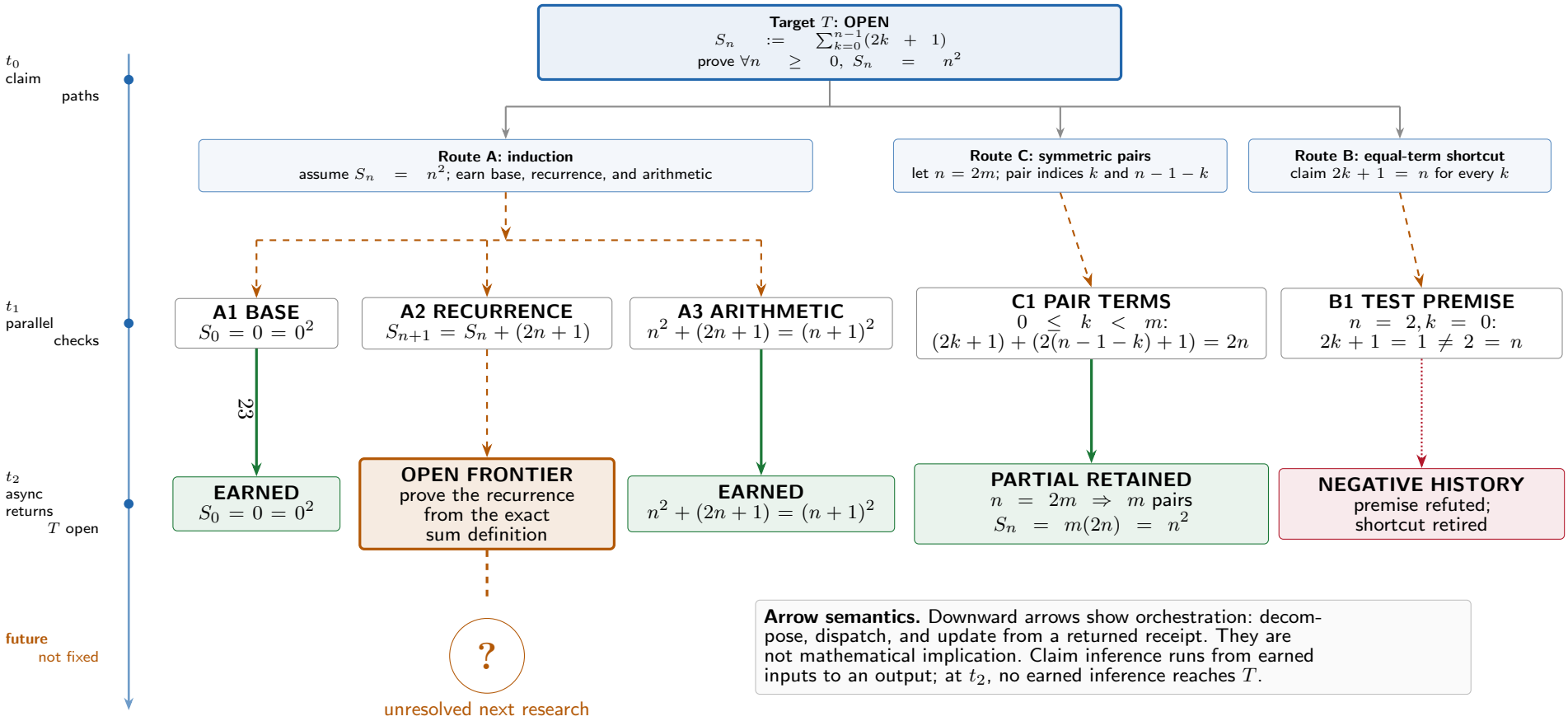


**Fig. 1** The operational projection maps transient agent work onto persistent claim state. Route A dispatches the base case, recurrence, and arithmetic identity; Route C computes symmetric pairs for $n = 2m$; Route B tests and refutes its termwise premise. Downward arrows represent orchestration events, not inference direction. At $t_2$, the recurrence is the sole open frontier. The broken continuation and question mark denote unresolved research and carry no earned receipt. Until a future earned route reaches $T$, the target remains open; the exact even-scope theorem and the failed shortcut remain visible.

The example separates truth from route status. The displayed identity is elementary and known to be true. Even so, the graph may not borrow that external knowledge to rehabilitate an invalid argument or fill an open edge. A correct partial theorem remains useful and earned at its actual scope. The figure intentionally leaves $T$ open because no complete earned route reaches the declared consumer in the recorded snapshot.

The vertical time spine is shared external state. One agent's memory has no persistent-state role. Earned, open, partial, and retired branches survive individual worker contexts, while only bound returns change the state seen by later workers. An open frontier names a selected obligation, not positive evidence. The question mark likewise names no predetermined step: it preserves the choice among further proof, decomposition, route replacement, and retirement. Only a future checked and currently bound result could change the recurrence edge; without it, both that edge and $T$ remain open.

The separation of arrow types is essential. The operational view moves from a target to dispatched work and then to returned state. The inferential graph has the opposite logical orientation: premises feed a hyperedge whose output is a larger claim. Because the recurrence receipt is missing, the operational view contains no active inference edge from Route A to $T$.

The example has supplied the concrete states and their consequences. The next subsection turns the remaining question, why one checked return may propagate while another may not, into an explicit promotion rule.

## 2.7 When evidence may carry an inference

A graph alone would merely automate overstatement if every locally checked edge could propagate. The method therefore needs a promotion gate between evidence that helps research and evidence that may carry an inference. The distinction between candidate evidence and an earned edge is the central inferential boundary of the framework.

**Definition 14** (Admissible promotion)**.** *Promotion of an edge $e$ from candidate to earned is admissible only when three groups of conditions hold. First, the output has reference grounding, an accepted machine-checkable artifact, and exact claim binding. Second, composition preserves meaning and strength, covers the complete obligation, and carries every assumption into the conclusion. Third, the evidence receipt matches the current semantic subject, definition closure, artifacts, checker, dependencies, and acceptance policy.*

Let $G(e)$, $M(e)$, $B(e)$, $S(e)$, $C(e)$, $D(e)$, and $I(e)$ denote, respectively, reference grounding, machine acceptance, claim binding, strength preservation, compositional completeness, dependency preservation, and current evidence identity. The abstract promotion rule is

$$e \in \mathcal{E}_{\text{earned}} \quad \Longleftrightarrow \quad G(e) \wedge M(e) \wedge B(e) \wedge S(e) \wedge C(e) \wedge D(e) \wedge I(e).$$

The right-hand side is conjunctive: satisfying only a proper subset leaves the edge in a candidate state. Table 4 states the interpretation of these predicates.

The seven terms protect three stages. $G$, $M$, and $B$ concern the object and its local check. $S$, $C$, and $D$ concern the inference into a larger claim. $I$ asks whether the same support is still current. The formula is one gate viewed at three levels; the seven terms remain jointly dependent.

The conditions address distinct logical failures. Reference material alone proves no new result. A mechanically accepted artifact may encode the wrong object or have a narrower scope than the claim that cites it. A correct local lemma may leave cases uncovered. A quantitative argument may count the same contribution twice. A conditional result cannot become unconditional by omitting its condition. Each defect blocks promotion. None contributes a fraction of closure.

| Condition | Theoretical requirement |
|---|---|
| Reference grounding $G(e)$ | The value or structure is identified by its definitions, normalization, reference inputs, provenance, and applicable baseline cases. |
| Machine acceptance $M(e)$ | A versioned artifact passes an independently executable checker under a declared trust and dependency policy. |
| Claim binding $B(e)$ | The grounded object and the checked object express the exact output claim at the required scope. |
| Strength preservation $S(e)$ | The statement established by each component implies the statement consumed by its parent under the same definitions and normalization. |
| Compositional completeness $C(e)$ | The components cover the whole obligation, and no case, endpoint, overlap, or quantitative contribution is silently omitted or counted twice. |
| Dependency preservation $D(e)$ | Every mathematical, computational, or literature assumption used by the components remains attached to the output claim. |
| Current evidence identity $I(e)$ | The receipt matches the active semantic subject, definition contracts, artifacts, checker, dependencies, and promotion policy; no load-bearing element has been invalidated or silently rebound. |

**Table 4** The candidate-to-earned rule is an all-or-nothing condition on inferential use. Candidate promise is recorded outside this rule.

**Principle 6** (All-or-nothing promotion)**.** *An edge becomes earned only when every condition required for its inferential use holds simultaneously. Otherwise it remains a candidate, regardless of how many conditions have already been established.*

The evolving graph supplies a compact test of this rule. Route A earns its edge only when the base, recurrence, and algebra inputs are jointly available. Finishing two of the three in parallel leaves the edge unearned and moves the active frontier to the remaining recurrence check. Route C also earns an edge whose output is $T_{\text{even}}$; the universal target $T$ remains open. Route B earns nothing because the $n = 2$ audit refutes its load-bearing premise.

### 2.7.1 Research closure and publication closure

An earned edge may establish a conditional research result while retaining an explicit external assumption. The graph can record this advance without describing it as an unconditional theorem. Publication closure is stronger: it also requires agreement among the public wording, evidence, assumptions, and verification boundary. Separating the two states prevents a locally correct component from being advertised as closure of a stronger public claim. Publication closure belongs to a claim and its public projection. The lifecycle state of the earning edge is a separate object.

## 2.8 Correction, invalidation, and re-earning

The reachability rule in Section 2.6 is monotone only while the semantic subject and evidence basis are fixed. Real research is not monotone in that sense. A later audit may find that a theorem was proved for the wrong carrier. It may find a missing endpoint or a failed transport between normalizations. It may instead find that a machine receipt used a stale dependency. In each case, the infrastructure must retract current inferential credit without rewriting the historical record.

**Definition 15** (Research snapshot)**.** *A research snapshot at time $t$ is*

$$\Omega_t = (\mathcal{G}_t, R_{0,t}, \Delta_t, \mathcal{R}_t, P_t),$$

*where $\mathcal{G}_t$ is the claim graph, $R_{0,t}$ the admissible root registry, $\Delta_t$ the active definition and semantic-contract registry, and $\mathcal{R}_t$ the evidence receipts. The final component, $P_t$, is the current acceptance policy. The earned edge set $\mathcal{E}_t^+$ consists exactly of edges having a current admissible receipt under $(\Delta_t, P_t)$.*

In less compact language, a snapshot is the graph and the rules that are current at one moment. It tells the reader which roots and definitions are active, which receipts exist, and which policy decides whether they count.

**Proposition 2** (Snapshot monotonicity)**.** *Fix the graph structure, root registry, definition registry, and acceptance policy* $(\mathcal{G}, R_0, \Delta, P)$*. Enlarging the receipt set by one admissible receipt, without removing an existing receipt or changing its admissibility, cannot decrease the resulting reachable set.*

*Proof* The reachability operator adds outputs of earned edges whose inputs are already reachable. The new receipt can only enlarge the earned-edge set. It removes no edge and no reachable claim at any iteration. The least fixed point therefore cannot shrink. □

Snapshot monotonicity is useful but insufficient. It explains why a sequence of valid new deductions accumulates. It provides no justification for keeping an edge earned after the subject, artifact, or policy that licensed it has changed.

**Definition 16** (Correction event)**.** *A correction event is a reviewed transition* $\Omega_t \to \Omega_{t+1}$ *that changes a load-bearing semantic contract, root claim, artifact, checker, dependency, route classification, or acceptance policy. The transition must identify every receipt whose evidence identity is affected and mark the corresponding edge invalidated unless a current admissible receipt already exists for the corrected subject. A root-registry change must also recompute assumption-relative closure and the corresponding publication labels.*

**Principle 7** (Fail-closed re-earning)**.** *Invalidation removes inferential credit immediately. The same edge may become earned again only through a new machine check and review bound to the corrected evidence identity. Recomputing a digest, renaming an object, or restating the old acceptance assertion is not re-earning.*

**Proposition 3** (Correction safety)**.** *Let $T$ be reachable at snapshot $\Omega_t$. Suppose the admissible root registry is unchanged, every route from those roots to $T$ in $\mathcal{G}_{t+1}$ contains an edge $e$, and a correction invalidates $e$. If no current admissible receipt re-earns $e$, then $T$ is not reachable at $\Omega_{t+1}$.*

*Proof* After invalidation, $e \notin \mathcal{E}^{+}_{t+1}$. By hypothesis every route to $T$ contains $e$, so the earned subgraph contains no complete route from the admissible roots to $T$. Derived closure therefore excludes $T$. □

The unchanged-root and all-current-routes conditions are load-bearing. A new earned route that avoids $e$ may preserve $T$, and a root change creates a new assumption-relative snapshot. Correction safety withdraws exactly the closure that depended on the invalidated edge while leaving independent evidence intact.

The safety property is deliberately stronger than a dashboard warning. A correction changes the mathematical state computed by the graph. Historical receipts remain available for audit, priority, and diagnosis, but they do not remain in the current earned set.

Route retirement requires a related polarity rule. Positive evidence for a route says that its output follows under its stated inputs. A retirement record says that the route is invalid, insufficient, superseded, or no longer relevant. The positive receipt cannot be reused as evidence for the negative claim. Retirement needs its own reviewed reason or counterevidence. It remains edge-relative. Retiring one proof leaves a conclusion with other possible incoming routes undecided.

Route B in Figure 1 is retired because the shortcut never earned inferential force. Invalidation applies to previously earned evidence. Now suppose an earned induction receipt is found to use a changed definition of $S_n$. The same tree removes that edge from current reachability. The edge returns only after a fresh check re-earns it.

This correction model also constrains supersession. A new definition may replace an old one for current work. The system must record how the two are related. They may be equal, conditionally equivalent, linked by a one-way implication, or merely historical successors. Only an earned transport can move evidence across that boundary. This prevents a convenient new object from inheriting status from a superficially similar predecessor.

## 2.9 Invariant core and domain-adaptive realizations

The method so far defines the inferential core. It explains how a candidate becomes earned, how earned claims compose, and how correction withdraws current status. The next question is what remains fixed across fields. Different research problems will use different tools, decompositions, and workflows. The evidentiary contract remains fixed. Claims are typed. Positive results require grounding, a check, and exact binding. Composition preserves strength and assumptions. Closure is derived from the record.

The architecture can in principle be parameterized by its *epistemic object*. The present evidence concerns mathematics. A future realization in another discipline might take an experimental measurement, statistical estimate, or verified program as its object. Each kind of object has its own account of evidence, error, replication, and closure. Even within mathematics, a finite classification, a symbolic identity, an asymptotic estimate, and a proof-assistant theorem generate different verification obligations.

Table 5 separates the framework invariants from their domain-adaptive realizations.

Engineering examples make the distinction concrete. A finite combinatorial classification may be organized around exhaustive case generation, versioned reference instances, compact witnesses, and a kernel-checked characterization. A mixed analytic–computational problem may instead require reference datasets, encoders, certificates, independently checked witnesses, analytic lemmas, and an explicit inventory of residual

| Framework invariant | Domain-adaptive realization |
|---|---|
| Typed claims and proof edges | The granularity of claims, the meaning of an edge, and the admissible decomposition of the target |
| Positive-evidence obligation | The form of reference data, the machine-checking technology, and the bridge connecting the checked object to the intended claim |
| Declared verification boundary | Witness checking, exhaustive search, certificate checking, symbolic validation, kernel checking, replication, or another domain-appropriate procedure |
| Candidate-to-earned admissibility | The concrete tests needed for semantic identity, coverage, quantitative accounting, and dependency preservation |
| Functional separation | The allocation of generation, verification, composition, and publication work among humans and machines |
| Publication closure | The public artifacts, metadata, assumptions, and replay instructions required for independent inspection |

**Table 5** The evidentiary invariants remain fixed while the engineering realization follows the research object.

assumptions. Highly branching proof search may benefit from a finer claim graph, systematic negative evidence, recursive strength checks, and quantitative non-duplication checks. These are adaptive choices; no component is mandatory across every proof engine.

## 2.10 Choosing the real research frontier

A claim graph can be logically accurate and still misdirect research. One route to a target may be exact but long. Another may be shorter but strictly stronger. A familiar route may already be invalidated, while an independent route may lack its inputs. Counting edges will not distinguish these cases. Nor will counting formalized components. Either shortcut can replace the real mathematical bottleneck with a different problem.

**Definition 17** (Research-base classification)**.** *Relative to a declared target and its exact active consumer, each route is described by every applicable relation below; the categories may overlap. In particular, architectural independence is orthogonal to the relation between a route's output and the exact consumer.*

*Exact.*

*Its earned output discharges the current consumer without strengthening or changing the target.*

*Supporting.*

*It supplies an input used by an exact route but is not itself a complete target route.*

*Partial.*

*It earns the intended statement on a strictly narrower scope but need not supply an input to the active exact route.*

*Strictly stronger.*

*Its contracted output strictly implies the consumer, but the additional strength is not required by the target. It may discharge the consumer only through an earned transport.*

*Independent.*

*It attacks the same target through a distinct proof architecture.*

*Negative.*

*It records a reproducible obstruction, counterexample, or retired route.*

*Historical.*

*It documents a superseded target, object, or research state and carries no current inferential or allocation priority.*

The classification leaves earned reachability untouched. It changes only the view used for reporting and resource allocation. Declaring an active frontier is therefore a

mathematical judgment. It names the minimum exact consumer the project is trying to produce now. A stronger route may remain valuable. A shorter graph path or a more mature implementation gives it no automatic priority.

**Principle 8** (No silent target substitution)**.** *A route may replace the declared active frontier only when an earned equivalence or implication shows that it discharges the same consumer at no weaker scope, or when a reviewed strategy transition explicitly changes the target. Graph topology, proof length, and implementation maturity are insufficient.*

The running graph also gives a minimal frontier transition. The base and algebra checks return earned. Route C earns $T_{\mathrm{even}}$, and Route B retires. The recurrence identity is now the sole unresolved input on the exact route to $T$. It therefore becomes the active frontier. Route C is a partial result at even scope, and Route B is negative knowledge. Both leave that missing input open.

Frontier discipline also constrains decomposition. The parent consumer fixes what a child producer must return. That contract includes the carrier, quantifiers, endpoints, normalization, sign, multiplicity, and assumptions. A convenient child may be stronger in some formal order and still fail to serve the parent. Its transport to the consumer must itself be earned.

This rule separates mathematical closure from search management. Closure is still derived from earned edges. The active frontier says where the next useful edge must enter that closure. Keeping the two concepts distinct prevents a project from appearing closer to completion merely because it accumulated formal work on an optional route.

Once the frontier fixes *what* must be established next, the remaining engineering question is *how* to verify it without weakening the claim. The next subsection states the cost objective; the following subsection then shows how formal proofs, certificates, witnesses, and replicated computations are allocated edge by edge.

## 2.11 Verification-cost minimization

**Principle 9** (Verification-cost minimization)**.** *Among architectures that preserve the required evidentiary status, prefer the one that lowers the total cost of independent verification and composition.*

AI may lower the cost of generating candidate outputs faster than the cost of verifying them [14]. For mathematical research, this asymmetry creates a concrete design objective: minimize the cost of independently checking and composing claim-bound evidence. At the level of one claim, the companion paper's economy-of-verification principle selects the least burdensome adequate check [16]. At the architectural level, however, local choices interact. An individually cheap check may require an expensive semantic bridge. It may introduce an incompatible representation. It may also force every parent claim to reverify the same dependency.

For a claim $C$ and an adequate boundary $b$, let $V_b(C)$ denote the cost to an independent reader of checking the corresponding evidence. Let $K(e)$ denote the additional cost of establishing that the inputs of edge $e$ are jointly sufficient for its output. For a declared target $T$, an architecture seeks a reachable earned subgraph $\mathcal{G}_T$ with low total cost

$$\sum_{C \in \mathcal{C}(\mathcal{G}_T)} V_{b(C)}(C) + \sum_{e \in \mathcal{E}(\mathcal{G}_T)} K(e),$$

subject to the admissibility conditions in Section 2.7. The expression is a design objective defined without assuming a universal numerical scale for verification effort.

The two sums represent different work. The first is the cost of checking the individual claims. The second is the cost of showing that those claims fit together. A locally cheap checker can therefore lead to an expensive proof architecture when composition is difficult.

Even this elementary evolution shows why adequacy constrains cost. The five checks can run concurrently because their outputs are independently typed. The single

instance $n = 2$ cheaply retires Route B but cannot verify $T$. Pairing cheaply proves $T_{\text{even}}$, but promoting it to the universal target would save checking effort only by changing the claim. Composition waits for the recurrence receipt because the base and algebra receipts alone do not reach the declared consumer.

The second term is essential. A collection of locally checked lemmas can remain difficult to assess when their definitions differ, their scopes do not align, or their assumptions are not propagated. Conversely, a moderately expensive proof object may lower total cost by eliminating repeated checks. Verification cost belongs to the composed architecture, not merely to an individual checker.

Time changes the order in which boundaries should be purchased, not the evidentiary standard for an earned edge. During exploration, a cheap witness or small-instance check may be the rational first investment because it can retire a path quickly. An edge may later become the active frontier. It may be consumed by many parents or be likely to survive into publication. At that point, a formal endpoint or durable certificate may have lower lifetime cost. An interim check may guide allocation while the edge remains candidate. A deadline cannot promote it at a weaker scope. This separates staged engineering from evidentiary dilution.

## 2.12 Allocating verification boundaries

The paired case paper compares concrete verification boundaries, including witness, certificate, kernel, dependency, and archival checks [16]. The architectural problem here is their allocation across a heterogeneous claim graph.

A boundary allocation is a partial map $b : \mathcal{C} \to \mathcal{B}$, where $\mathcal{B}$ is the set of checking procedures available in the relevant domain. The allocation is valid only when $b(C)$ is adequate for $C$. Its output must also enter every consuming edge without changing the claim or dropping assumptions. Different claims in the same argument may therefore use different boundaries. Each record names the accepted inputs and exact output.

It also names the checker, acceptance predicate, trust assumptions, and downstream consumer. A green solver status without this contract fails to qualify as a claim receipt.

No boundary is privileged independently of the proof object. Kernel checking may suit a formal theorem. Certificate checking may suit a finite refutation, and direct validation may suit a witness. A computational or empirical object may require independent replication. The architecture must make these differences composable. Changing the epistemic object may change both the available boundaries and the shape of the claim graph.

Table 6 makes the project dependence explicit. The rows are alternatives and combinations; each project selects the applicable modules.

| Project condition | Adequate edge evidence and binding | Time and reuse rule |
|---|---|---|
| Object-sensitive representation or parsing | Versioned parser or independent decode, definition contract, source-to-object checks, and a binding from parsed object to the exact consumer | Invest early when endpoints, multiplicity, normalization, or carrier identity can change the theorem |
| Long symbolic or compositional proof | Kernel-checked endpoint plus theorem map that records imports, assumptions, statement type, and paper consumer | Formalize load-bearing interfaces and reusable producers before peripheral lemmas |
| Large finite or SAT-heavy residual | Instance-bound SAT/UNSAT or domain certificate, independently replayable checker, decoded proposition, and complete coverage record | Certificate work becomes worthwhile when search output is load-bearing; solver success alone remains candidate |
| Cheap falsification or explicit witness | Independently reproducible instance, predicate evaluation, and binding to the route it retires or the scoped claim it earns | Use early when one small check can remove an expensive path without pretending to verify the target |

**Table 6** Verification boundaries are allocated by mathematical object, claim role, and lifetime checking cost. Every row still requires exact input-output binding.

## 2.13 Consequences for research state and publication

The core machinery is now in place. This subsection introduces no new graph mechanism. It collects the reporting rules that follow when local evidence, composition, correction, and frontier choice are considered together.

**Principle 10** (Local positivity is not global closure)**.** *A checked local result changes the status of the target only when it enters an earned edge on a complete route from the admissible roots.*

A formalized lemma, an exhaustive computation on a proper subdomain, or a persuasive conditional argument therefore cannot be mistaken for the stronger claim that consumes it.

**Principle 11** (Negative evidence is bound to what it refutes)**.** *Negative evidence changes only the proposition to which it is exactly bound. Refuting a route premise retires that edge without refuting its proposed output. A verified counterexample to the output claim instead refutes that claim at the counterexample's stated scope.*

A route-local failure can remain valuable when its assumptions, method, and obstruction are reproducible. Other incoming edges may still establish the same output claim. By contrast, a claim-bound counterexample blocks publication of that claim at the refuted scope and requires any conflicting earned route to be invalidated or reviewed. The binding prevents a failed method from becoming a false theorem and prevents a genuine counterexample from being demoted to a mere route failure.

**Principle 12** (Receipts are subject-relative)**.** *A machine receipt licenses only the semantic subject, definition closure, artifacts, dependencies, and policy to which it was bound. Byte identity alone cannot transfer the receipt to a corrected subject.*

**Principle 13** (Open obligations cannot be absorbed by description)**.** *A positive composite edge must inventory every load-bearing assumption, error term, case, factor opening, and transport required by its output. Naming an obligation, bounding one component, or recording a conditional consumer does not discharge it.*

This principle generalizes beyond analytic estimates. A finite certificate cannot omit a residual instance. A case classification cannot omit a boundary case. A formal theorem cannot hide a project axiom. A literature theorem cannot cross to a new normalization by citation alone. Completeness is a typed property of the exact consumer.

**Principle 14** (Closure-respecting publication)**.** *The evidentiary status stated in a publication must be the image of an earned subgraph under a projection that preserves claims, assumptions, and verification boundaries.*

Publication may omit exploratory history, alternative routes, and internal coordination records, but it must not strengthen the surviving claims or erase their dependencies. The companion paper develops the corresponding claim-level publication and reproduction procedure; this principle states the graph-level constraint on that procedure [16].

## 2.14 Relationship to existing methods

Proof Engine Infrastructure brings several older and newer lineages into a distinct transition architecture for mathematical research. Truth-maintenance systems maintain alternative assumption environments and withdraw conclusions when their justifications fail [4]. Assurance cases structure claims, arguments, defeaters, and supporting evidence [5]. W3C PROV records entities, activities, agents, revision, and invalidation, while RO-Crate packages connected research artifacts and their machine-readable provenance [6, 7]. The EVI ontology goes further by representing support and challenge relations

across scientific evidence graphs [8]. These are direct antecedents for justification maintenance, structured argument, and provenance-aware correction.

Proof Engine makes a different object operational. Its persistent state is an exact mathematical claim graph. Belief sets, assurance diagrams, and artifact catalogs remain separate objects. Agents are transient workers attached to typed claim obligations. A return changes state only through an adequate checker, an exact input-output binding, an admissible root and dependency policy, and an earned hyperedge. Correction recomputes current mathematical reachability, while the active frontier dispatches the next exact consumer. This conjunction is the agent-to-claim control architecture; provenance and graph structure are jointly necessary and individually insufficient.

Recent agent-native research systems provide a closer operational comparison. Agent-Native Research Artifacts organize claims, executable code, raw evidence, and an exploration graph with retained dead ends so that later agents can understand and extend a research package [9]. Argus uses a Navigator and parallel Searchers to assemble a shared evidence graph for source-traced deep-research answers [10]. Both make persistent research state useful to agents. Proof Engine addresses a different transition: it decides whether a generated mathematical object may carry an inference into an exact theorem consumer, and it keeps that decision current after correction.

Evidence-graph agents are closer still. VeriGraph constructs an execution-time DAG from raw data and computations to natural-language claims in data analysis [11]. EviGraph makes a typed evidence graph the operational state of an autonomous empirical-research agent and repairs weak downstream subgraphs through checkpointed regeneration [12]. Those systems contribute implementations and quantitative benchmarks for data and experimental workflows. Proof Engine contributes a fail-closed mathematical state semantics across formal proofs, certificates, witnesses, literature inputs, and publication, together with three completed mathematical cases.

| Method line | Persistent research object | Control boundary relative to Proof Engine |
|---|---|---|
| Truth maintenance, assurance, and provenance | Assumption environments, structured arguments, entities, activities, and evidence relations | Maintains justification or provenance; by itself leaves mathematical work-unit assignment and exact machine-bound theorem consumers unspecified |
| Agent-native artifacts and evidence assembly | Research packages, exploration traces, sources, and answer evidence | Makes research state agent-readable and dispatchable; by itself leaves earned mathematical closure and re-earning unspecified |
| Data and empirical evidence-graph agents | Execution DAGs and evolving experiment-to-claim evidence | Supplies graph construction, repair, and benchmarks in its task domain; leaves heterogeneous mathematical verification boundaries outside a single theorem state |
| Agentic theorem proving | Formal goals, proof states, repositories, and checked proof attempts | Optimizes proof generation and local formal acceptance; does not govern the whole paper-to-evidence-to-publication research state |
| Proof Engine Infrastructure | Exact claims, obligations, roots, receipts, definition contracts, and frontier state | Maps replaceable agents to claims; within a fixed root snapshot, only current bound evidence and earned inference may alter theorem closure |

**Table 7** Proof Engine is distinguished by the controlled relation from replaceable agents to exact claims and from claim-bound evidence to current mathematical closure.

The distinctive contribution is this combined architecture and its demonstrated mathematical closure record. The public Proof Engine formulation, archived on 29 July 2026 [13], integrates admissible-root governance, typed claim-path work units, heterogeneous evidence receipts, earned AND/OR composition, correction and re-earning, exact-consumer frontier selection, and closure-respecting publication in one agent-to-claim control plane. The three completed deployments show the same control plane operating across different mathematical objects and verification regimes.

Table 7 summarizes the boundary between Proof Engine and the adjacent method families.

Work on neural and LLM-based proof generation began with generative theorem proving [36]. It now includes retrieval-augmented and large-scale systems [39, 40], as well as competition-level systems [25, 41]. Newer agentic systems expose planner, worker, repository, repair, and verifier functions explicitly [26, 27]. These systems target proof generation and local formal acceptance. Proof Engine can consume their checked returns without inheriting their agent topology or treating their success signal as graph-level authority.

Autoformalization translates informal statements and proofs into a proof assistant [37, 38]. Proof assistants and their libraries then supply a kernel-checking boundary [28, 29]. The translation remains generated until it has been checked and its assumptions audited. Kernel acceptance is powerful, but Proof Engine must still bind the accepted theorem to the paper claim and compose it with any other checked evidence.

Large formal developments also create software-like problems of organization, abstraction, build cost, evolution, and maintenance. Proof engineering systematizes these concerns, while proof repair studies how developments survive changes in types and specifications [23, 24]. Proof Engine addresses the prior semantic decision: does the changed object still denote the claim consumed by the research graph, and does earlier evidence remain admissible?

SAT-based mathematics and certified checking [30–35] supply finite and certificate boundaries. In a SAT-heavy path, the solver is a generator unless its result is accompanied by an independently replayable proof or certificate. A graph-level receipt must bind the encoded input, certificate, decoded proposition, coverage scope, and exact downstream edge.

Adjacent papers in the present program locate the same bottleneck at different scales. Verification Asymmetry explains why candidate generation can accelerate while verification capacity becomes scarce [14]. The mathematical-engineering framework follows a closed result into explanation, reuse, resource profiles, stable interfaces, and

lifecycle ownership [22]. Proof Engine occupies the live transition between them. Its contribution is the combined agent-to-claim loop: admissible roots, typed work units, heterogeneous receipts, earned AND/OR composition, correction and re-earning, exact-consumer frontier selection, and closure-respecting publication, demonstrated across completed mathematical workflows.

# 3 Results

## 3.1 Three successful projects and two closure regimes

The comparison asks one methodological question. How do the same control rules earn the right target in different successful mathematical workflows? Before answering, each project must be understandable as mathematics. Internal repository names carry no explanatory role. Table 8 gives the reader-level map. The following subsections then explain each project's problem, architecture, and outcome.

| Project question | Mathematical route and Proof Engine stress | Outcome |
|---|---|---|
| Which noncrossing-partition refinement graphs have Hamilton cycles or paths? | Uniform construction and parity obstruction must bind to the exact Lean classification endpoints | Exact classification CLOSED |
| What is the exact extremal value in Erdős Problem 848 for every $N$? | Exact Hall equivalence, structural compression, exhaustive certificates, and the uniform tail compose under semantic checking and kernel replay | Exact Erdős Problem 848 solution for all $N$: CLOSED |
| Can equal periodic Lagrange values determine band-graph isomorphism, and how are cover relations characterized? | Exact counterexample and cover certificates bind to premise-free formal endpoints | Direct closure pattern successfully reused |

**Table 8** The mathematical question comes first; the closure regime describes which part of the method the successful project tests.

### 3.1.1 Closed case: a uniform Hamilton classification

The first project studies the cover graph of the noncrossing-partition refinement lattice. Its vertices encode noncrossing partitions, and its edges record cover relations in refinement order. If a one-vertex graph is regarded as Hamiltonian through its constant closed spanning walk, then across all natural $n$ the graph has a Hamilton cycle exactly when $n \in \{0, 1\}$ or $n \geq 4$ is even. Thus, in the usual nondegenerate range $n \geq 3$, it has a Hamilton cycle exactly when $n$ is even. Across all natural $n$, it has a Hamilton path exactly when $n \leq 3$ or $n$ is even [15].

These classifications and their proofs are the new mathematical result of the project. The Lean formalization supplies matching endpoints. The formal development was built with the mathematical argument and supplies matching endpoints.

The proof architecture is close to the mathematical explanation. The even case organizes vertices into Boolean blocks. A Gray-code path traverses each block, and local switches join the blocks. A signed block-parity argument rules out the missing odd cases. The Lean development follows the same decomposition and ends in premise-free classification endpoints.

This project tests exact binding. Reference grounding fixes the graph and its adjacency relation. It also fixes the range of $n$ and the endpoint conventions. Kernel acceptance checks the formal endpoints. A theorem map then binds those endpoints to the two paper classifications. The construction, formal statement, and declared consumer agree, so the classification target is CLOSED.

### 3.1.2 Closed case: exact all-$N$ solution of Erdős Problem 848

The second project asks for the exact value in Erdős Problem 848. If $A \subseteq [1, N]$ and $ab + 1$ is nonsquarefree for every $a, b \in A$, including $a = b$, the project proves

$$|A| \leq \left\lfloor \frac{N + 18}{25} \right\rfloor,$$

with equality attained by the progression $7 \pmod{25}$, for every positive $N$ [17].

Sawhney proved that the class $7 \pmod{25}$ is extremal for all sufficiently large $N$. Sothanaphan later made the asymptotic inputs explicit and obtained the threshold $N \geq 2.64 \cdot 10^{17}$. The range below that threshold remained open [19, 20]. An OpenAI report publicized this asymptotic advance under the heading "AI-assisted solution to Problem #848." The same report states the sufficiently-large-$N$ scope [18]. The project studied here solves the original exact problem for every positive $N$. It is new mathematics with a matching kernel-accepted endpoint and extends beyond a Lean transcription of the asymptotic result.

The proof combines structural compression and exact finite mathematics. An exact Hall equivalence replaces the extremal-set question by a completion inequality. Exact prefix colourings close $N \leq 5 \cdot 10^6$. Beyond that range, square-divisor estimates and a second mod-25 construction reduce the possible Hall defects. The proof then separates five 2-adic types and residue classes modulo 9. Three- and four-pivot arguments lead to finite-prime counts and counts of roots of a transformed quadratic equation. Exact rational inequalities cover the remaining finite cases. A uniform estimate handles the unbounded tail. The finite calculations prove exact inequalities under general soundness theorems. They are mathematical parts of the all-$N$ proof and carry no extrapolation from sampled values.

The Proof Engine route treats certificate generation as untrusted. Generated outputs first pass semantic checkers that verify their intended finite meaning and coverage. Heterogeneous receipts then connect the mathematical reductions, finite certificates, and Lean endpoint. The composed route earns the exact all-$N$ theorem. Its large transitive replay burden remains a separate engineering problem; the mathematical premise is closed [22].

### 3.1.3 Successful reuse: rational Dyck paths

A later rational-Dyck-path project asks two questions. Do equal periodic Lagrange values force the associated band graphs to be isomorphic? How can cover relations in the matching and Lagrange orders be characterized? The project supplies an exact minimal counterexample to the first question, constructive cover certificates, and premise-free formal endpoints [21].

The counterexample lies in $\mathcal{D}(17, 9)$ and therefore has total length 26. Two independent exact enumerations exclude counterexamples through total length 25. Prefix-antichain certificates characterize the cover relations, and Lean replays the numbered results. The Proof Engine role is to bind the exact counterexample and its finite minimality evidence. It also binds the structural theorems to premise-free endpoints. Enumeration can then support the result without becoming an unchecked premise.

The project successfully reuses the direct mathematical-to-kernel release pattern on a new object. It supplies repeatability evidence while remaining in the same closure regime as the Hamilton case.

### 3.1.4 Validated function-level behaviors

The deployments also test smaller functions inside the closed loop. Their usefulness is conditional on the project type; success in one project leaves the function optional elsewhere.

Table 9 records the function-level behaviors observed across the completed projects.

| Function observed | Evidence in the completed projects | Where it is useful |
|---|---|---|
| Object-sensitive decoding and semantic checks | Finite certificates were decoded back to their intended instances and coverage claims before entering the all-$N$ route | Encoded, parsed, normalized, or endpoint-sensitive objects; less consequential when the object is already canonical and literal |
| Paper-to-kernel theorem mapping | Hamilton and rational-Dyck endpoints were matched to the exact public classifications, counterexample, and cover claims | Formal developments whose theorem names or nearby stronger statements could otherwise be mistaken for the paper consumer |
| Certificate replay and coverage accounting | The extremal theorem separated untrusted generation from semantic checking, complete finite coverage, and kernel consumption | Large finite and SAT-like residuals where a solver answer is too large or opaque for direct inspection |
| Mixed witness and structural closure | The rational-Dyck project combined an exact counterexample, two minimality enumerations, cover certificates, and premise-free endpoints | Problems where one claim is existential while another requires exhaustive or structural closure |

**Table 9** Observed functions have project-dependent value inside the same minimum closed loop.

A SAT-dominant project would instantiate the third row with an instance-bound SAT or UNSAT certificate and an independent replay. The binding would run from the encoder input, through the decoded solver output, to the exact graph edge. A short constructive route may need no SAT layer at all. The invariant is the adequacy and composability of the receipt, independent of tool choice.

### 3.1.5 Operational stress evidence from concurrent development

The separate ongoing proof program tested the coordination layer under a wider range of live states than the three completed deployments. Parallel workers were used for proof-route exploration and for the formalization of distinct producers. Other workers performed adversarial review, correction, and later assembly. The comparison between

narrower and wider dispatch was operational and lacked a standardized task suite, token budget, hardware budget, and timing protocol.

The observed operational utility was conditional but direct. Independent work units advanced concurrently. Verified receipts survived the end of an agent context. Review or correction could invalidate one route without erasing unaffected work. The same use exposed real costs. Wide branching consumed substantial tokens. Competing workers sometimes approached the same obligation, and changed contracts triggered coordination and recheck work. These observations support the usefulness of the graph as a concurrency control plane. They do not establish net acceleration, a universal optimal worker count, or a quantitative speedup factor.

### 3.1.6 What the comparison establishes

The three projects address genuinely different mathematical questions and use two materially different closure architectures. Their successful completion provides bounded evidence that the method's claim-binding, composition, and closure rules can transfer across problem types within mathematics. The projects also share one research-program provenance. That limits independence of the evaluation, not the heterogeneity of the problems. Independent implementations remain necessary to establish cross-team portability and comparative productivity.

# 4 Discussion

The comparison places Proof Engine Infrastructure between a single-project feasibility claim and a universal software benchmark. The evidence is stronger than a toy demonstration because the same state rules were used in three different mathematical problems. It remains bounded because the projects share one research program and no independent team has yet implemented the method.

Each completed case illuminates a different part of the architecture. The Hamilton classification shows why kernel acceptance must be joined to an exact paper-to-theorem binding. Erdős Problem 848 shows how structural mathematics, exhaustive exact certificates, semantic checking, complete range coverage, and kernel replay can jointly earn one all-$N$ theorem. The rational-Dyck project shows that the direct mathematical-to-kernel pattern can be reused on a new object. The odd-sum graph then makes the shared state logic visible at small scale. A false route retires. A true even-scope theorem is retained. The universal target remains open at its exact missing input.

The ongoing stress case adds an operational interpretation. Typed work units allowed proof search and formalization to proceed concurrently. Review and correction could proceed in parallel as well. Receipts survived the end of individual agent contexts. When a return was stale, duplicated, or attached to the wrong scope, the graph could preserve the artifact without granting it new inferential force. This observed behavior is the practical reason the closed loop is more than a visual wrapper around a multi-agent harness.

The concurrency evidence is positive but qualitative. The graph exposed work that was already covered, preserved reusable verified results, and identified obligations independent enough to dispatch in parallel. Wider branching also consumed substantial tokens and coordination effort. The evidence therefore supports useful concurrent progress under suitable decomposition. Positive net speedup remains an open evaluation question in the present study.

The research-integrity implication is a division of labor between provenance and inference. Structured AI-use documentation records where and how a system entered the workflow [1, 2]. A claim receipt records what object was checked, what proposition was accepted, which consumer uses it, and whether that support remains current. These are separate questions. A responsible record needs both, and each component carries a distinct function.

Proof Engine also precedes the mathematical engineering layer that packages a closed result for explanation, reuse, resource planning, stable interfaces, and lifecycle ownership [22]. Its narrower task is to decide whether closure has been earned in the first place and how later correction changes that status. Keeping the layers separate prevents presentation quality or implementation sophistication from substituting for mathematical support.

## 4.1 Toward Proof Engine 2.0: trusted agent concurrency

The current method establishes a minimum claim-state loop and operational feasibility for concurrent development. The next research question concerns which graph-native protocol makes demonstrated parallel work efficient and trustworthy. A future Proof Engine 2.0 line will develop this protocol before exposing implementation-specific coordination mechanisms.

The protocol has at least six requirements:

1. every work unit carries an immutable semantic-snapshot identifier, exact consumer, prerequisite receipts, and output contract.
2. edge reservations or renewable leases prevent avoidable duplicate work while allowing abandoned work to return safely to the frontier.
3. agent-to-agent handoffs are typed graph events containing artifacts, evidence identity, scope, dependencies, and freshness; unstructured assertions of success carry no transition authority.
4. one agent may reuse another agent's result only through a receipt whose checking and promotion state satisfy the consuming edge; peer prose is never inherited as evidence.
5. divergent returns enter an explicit conflict, supersession, and independent-review procedure instead of being merged by confidence or recency.

6. dependency-aware scheduling, concurrency quotas, cancellation, and token/compute accounting optimize useful earned progress; raw worker activity is only an input measure.

This protocol also makes quantitative evaluation possible. Matched work units can be run serially, with narrow parallelism, and with wider parallelism. Each run can record wall-clock time, model tokens, and compute. It can also record duplicated work, accepted receipts, stale or conflicting returns, and recheck cost. A final measure asks which results survive correction. The aim is not maximal concurrency. It is the smallest communication and trust protocol that preserves the current fail-closed semantics while improving net research throughput. The implementation and controlled evaluation belong to future work; the invariants specified here are intended to remain its stable substrate.

# 5 Limitations

Three limitations define the available evidence.

First, Proof Engine Infrastructure is implementation-independent. It may be realized through an agent harness or an API-orchestrated workflow. Other research systems may implement it as well, and it need not be fully automated. The current implementation remains under active development and is not released as an unsupported public tool. The article instead specifies the behavioral invariants and reporting rules needed for independent implementation. An independent implementation may use different storage, agents, parsers, solvers, or proof assistants. It must still preserve the minimum loop. Reproducing only the graph vocabulary or the parallel dispatch surface would not test the method.

Second, the transfer evidence is bounded but nontrivial. The three deployments address distinct mathematical questions. They instantiate two closure architectures,

and one pattern was reused on a third object. The method is therefore not supported by one theorem alone. Because one research program produced all three, cross-team portability remains untested. The cost objective is also qualitative: there is no universal empirical scale for verification effort. Narrower and wider parallel configurations were compared during ongoing development, but not under a fixed task suite, baseline, token budget, hardware budget, or timing protocol. The study therefore reports practical usefulness and substantial resource cost. Net speedup, duplicate-work reduction, and containment of context loss or hallucination remain unestimated. The odd-sum tree illustrates state evolution and is excluded from the deployment count. Non-mathematical domains remain open evaluation targets. So do usability and correction behavior under independent implementations.

Third, semantic completeness still depends on mathematical judgment. A definition contract exposes the fields selected by its author. It cannot prove that every material field was selected. Root admission may also misclassify an unsupported claim as an assumption or imported result. Required tokens and source anchors detect known corruption modes, not meaning itself. An assumption or error inventory may still omit an obligation. Hostile review and independent formalization therefore remain essential. The method contributes a narrower guarantee: once a defect is found and the affected receipts are identified, the correction protocol revokes their current inferential status instead of leaving them active behind a prose-only warning.

# 6 Conclusions

A proof engine begins where candidate generation ends. It fixes the exact target and admissible roots, then decomposes that consumer into typed claim paths. It records what each local artifact establishes. Reference grounding, an adequate check, and exact claim binding create candidate evidence. All-or-nothing promotion and graph composition determine whether that evidence may reach the target. Formal proofs,

certificates, direct witnesses, and replicated computations are different edge boundaries. They are chosen for the mathematical object and retain non-interchangeable evidentiary meanings.

The same graph supplies an agent-to-claim control plane across workers and time. Typed work units allow independent obligations to proceed concurrently. Asynchronous returns enter through receipts; worker authority cannot substitute for them. Within a fixed semantic snapshot, valid evidence accumulates. After a correction, affected edges lose current reachability, historical receipts remain auditable, and re-earning requires a fresh check bound to the corrected object. Frontier discipline then keeps new work attached to the exact unresolved consumer.

The completed cases show that substantially different proof shapes can satisfy these rules. The mathematical construction, its kernel endpoints, and exact paper-to-kernel binding close the Hamilton classification. Structural mathematics, exhaustive certificates, semantic checking, and kernel replay close Erdős Problem 848 for every $N$. The rational-Dyck project reuses direct closure on a third object. The odd-sum example displays the same logic without borrowing the known truth of its target. The even-scope theorem is retained. The false shortcut retires. The precise recurrence frontier stays open.

Operational use shows that the graph can coordinate useful concurrent research, while wide dispatch can impose substantial token and communication costs. Proof Engine 2.0 will therefore study reservations, typed handoffs, receipt-only inter-agent trust, conflict resolution, scheduling, and resource accounting on matched work units. Until those evaluations exist, this article contributes the fail-closed state method. A publication should make no stronger claim than the earned graph supports. Its status should change when its evidence changes.

# Declarations

**Ethics approval and consent to participate.**

Not applicable. The study did not involve human participants, human data, animals, or identifiable personal information.

**Consent for publication.**

Not applicable.

**Availability of data and materials.**

All case observations needed to assess the qualitative comparison are included in this article. The public mathematical manuscripts and formal proof packages for the successful projects are available through the persistent identifiers cited in the reference list. The ongoing concurrency stress case is reported only at the workflow-observation level. Its open mathematical target and unreleased research materials are not part of the evidence for the three public results. The implementation remains under active development and is not distributed. No substantive mathematical conclusion depends solely on an output of that implementation.

**Competing interests.**

The author declares that he has no competing interests.

**Funding.**

No funding was received for conducting this study or preparing the manuscript.

**Authors' contributions.**

ACL conceptualized the study, developed the method and transition semantics, conducted the deployment analysis, and wrote and approved the manuscript.

**Acknowledgements.**

Not applicable.

# Additional file

**Additional file 1.** Plain-text file. *Proof Engine Infrastructure reporting and audit checklist.* A reusable claim-level checklist covering semantic subject, evidence, exact consumer, dependencies, status, correction, frontier, and publication label.

**Declaration of Generative AI and AI-assisted technologies in the writing process.**

During the preparation of this work the author used OpenAI Codex for manuscript preparation. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

# References


[1] Jan Christopher Cwik. Disclosure is not documentation: an open science framework for documenting generative AI use in scholarly research and publication workflows. *Research Integrity and Peer Review*, 11:47, 2026. 10.1186/s41073-026-00245-8.

[2] Neil Mehta et al. A call for clarity: a unified checklist for reporting use of large language models in writing scientific manuscripts. *Research Integrity and Peer Review*, 11:24, 2026. 10.1186/s41073-026-00212-3.

[3] Marcus R. Munafò et al. A manifesto for reproducible science. *Nature Human Behaviour*, 1:0021, 2017. 10.1038/s41562-016-0021.

[4] Johan de Kleer. An assumption-based TMS. *Artificial Intelligence*, 28(2):127–162, 1986. 10.1016/0004-3702(86)90080-9.

[5] John B. Goodenough, Charles B. Weinstock, and Ari Z. Klein. *Toward a Theory of Assurance Case Confidence.* Technical Report CMU/SEI-2012-TR-002, Software Engineering Institute, Carnegie Mellon University, 2012. 10.1184/R1/6585362.v1.

[6] James Cheney, Paolo Missier, Luc Moreau, and Tom De Nies. *Constraints of the PROV Data Model*. W3C Recommendation, 30 April 2013. w3.org/TR/2013/REC-prov-constraints-20130430/.

[7] Stian Soiland-Reyes et al. Packaging research artefacts with RO-Crate. *Data Science*, 5(2):97–138, 2022. 10.3233/DS-210053.

[8] Sadnan Al Manir, Justin Niestroy, Maxwell Adam Levinson, and Timothy Clark. Evidence graphs: Supporting transparent and FAIR computation, with defeasible reasoning on data, methods, and results. In *Provenance and Annotation of Data and Processes*, pp. 39–50, Springer, 2021. 10.1007/978-3-030-80960-7_3.

[9] Jiachen Liu et al. The last human-written paper: Agent-native research artifacts. arXiv:2604.24658, arxiv.org/abs/2604.24658, 2026.

[10] Zhen Zhang et al. Argus: Evidence assembly for scalable deep research agents. arXiv:2605.16217, arxiv.org/abs/2605.16217, 2026.

[11] Jiajie Jin et al. VeriGraph: Towards verifiable data-analytic agents. arXiv:2606.16603, arxiv.org/abs/2606.16603, 2026.

[12] Zhenjiang Ren et al. EviGraph: Evidence-guided autonomous research agents. arXiv:2608.04738, arxiv.org/abs/2608.04738, 2026.

[13] Alex Chengyu Li. *Proof Engine Infrastructure: A Claim-Graph Framework for AI-Assisted Mathematical Research*. Zenodo version DOI 10.5281/zenodo.21672334, 29 July 2026.

[14] Alex Chengyu Li. *Verification Asymmetry under AI Substitution: Wage-Ratio Inversion and Apprenticeship Thresholds*. Zenodo concept DOI 10.5281/zenodo.20038847, 2026.

[15] Alex Chengyu Li. *Hamilton Cycles and Paths in the Noncrossing Partition Refinement Graph*. SSRN DOI 10.2139/ssrn.7104338; technical archive concept DOI 10.5281/zenodo.21316750, 2026.

[16] Alex Chengyu Li. *Claim-Level Verification for AI-Assisted Mathematics: A Kernel-Closed Hamilton Classification and a Mixed-Boundary Rado Study*. SSRN 6892258; DOI 10.2139/ssrn.6892258, 2026.

[17] Alex Chengyu Li. *Erdős Problem 848: A Kernel-Checked Proof of the Exact Extremal Bound*. SSRN 7230480; DOI 10.2139/ssrn.7230480, 2026.

[18] OpenAI. *Early Science Acceleration Experiments with GPT-5*. OpenAI, Chapter IV.1 describes the Sawhney–Sellke result for Erdős Problem 848, 2026. OpenAI report PDF.

[19] Mehtaab Sawhney. *On $A \subseteq [N]$ Such That $ab+1$ Is Never Squarefree for $a, b \in A$*. Preprint, 2025. https://www.math.columbia.edu/~msawhney/Problem_848.pdf.

[20] Nat Sothanaphan. *An Explicit Threshold in Erdős Problem 848*. Preprint, 2026. https://drive.google.com/file/d/1ujhm4_WYpgRV_rd1rJXIfHyvx16COEKe/view.

[21] Alex Chengyu Li. *Lagrange Collisions and Cover Relations for Rational Dyck Paths*. Zenodo concept DOI 10.5281/zenodo.21980889, 2026.

[22] Alex Chengyu Li. *Beyond Kernel Verification: The Missing Engineering Layer of Mathematics*. SSRN 7237343; DOI 10.2139/ssrn.7237343, 2026.

[23] Talia Ringer, Karl Palmskog, Ilya Sergey, Milos Gligoric, and Zachary Tatlock. QED at large: A survey of engineering of formally verified software. *Foundations and Trends in Programming Languages*, 5(2–3):102–281, 2019. 10.1561/2500000045.

[24] Cosmo Viola, Max Fan, and Talia Ringer. Proof repair across quotient type equivalences. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):3177–3202, 2025. 10.1145/3763164.

[25] Tudor Achim et al. Aristotle: IMO-level automated theorem proving. arXiv:2510.01346, arxiv.org/abs/2510.01346, 2025.

[26] David Ma et al. OProver: A unified framework for agentic formal theorem proving. arXiv:2605.17283, arxiv.org/abs/2605.17283, 2026.

[27] Matěj Kripner and Milan Straka. OpenProver: Agentic and interactive theorem proving with Lean 4. arXiv:2607.09217, arxiv.org/abs/2607.09217, 2026.

[28] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, LNCS 12699, Springer, 2021, pp. 625–635. 10.1007/978-3-030-79876-5_37.

[29] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, ACM, 2020, pp. 367–381. 10.1145/3372885.3373824.

[30] Marijn J. H. Heule, Oliver Kullmann, and Victor W. Marek. Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer. In *Theory and Applications of Satisfiability Testing – SAT 2016*, LNCS 9710, Springer, 2016, pp. 228–245. 10.1007/978-3-319-40970-2_15.

[31] Marijn J. H. Heule. Schur number five. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18)*, AAAI Press, 2018, pp. 6598–6606. 10.1609/aaai.v32i1.12209.

[32] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt Jr. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014*, LNCS 8561, Springer, 2014, pp. 422–429. 10.1007/978-3-319-09284-3_31.

[33] Luís Cruz-Filipe, Marijn J. H. Heule, Warren A. Hunt Jr., Matt Kaufmann, and Peter Schneider-Kamp. Efficient certified RAT verification. In *Automated Deduction – CADE 26*, LNCS 10395, Springer, 2017, pp. 220–236. 10.1007/978-3-319-63046-5_14.

[34] Peter Lammich. Efficient verified (UN)SAT certificate checking. In *Automated Deduction – CADE 26*, LNCS 10395, Springer, 2017, pp. 237–254. 10.1007/978-3-319-63046-5_15.

[35] Stephan Gocht, Ciaran McCreesh, and Jakob Nordström. An auditable constraint programming solver. In *28th International Conference on Principles and Practice of Constraint Programming (CP 2022)*, LIPIcs 235, 25:1–25:18, 2022. 10.4230/LIPIcs.CP.2022.25.

[36] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. arXiv:2009.03393, arxiv.org/abs/2009.03393, 2020.

[37] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022. arxiv.org/abs/2205.12615.

[38] Christian Szegedy. A promising path towards autoformalization and general artificial intelligence. In *Intelligent Computer Mathematics (CICM 2020)*, LNCS 12236, Springer, 2020, pp. 3–20. 10.1007/978-3-030-53518-6_1.

[39] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. arxiv.org/abs/2306.15626.

[40] Huajian Xin et al. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. arXiv:2405.14333, arxiv.org/abs/2405.14333, 2024.

[41] Google DeepMind. AI achieves silver-medal standard solving International Mathematical Olympiad problems (AlphaProof and AlphaGeometry 2). Technical report / announcement, deepmind.google/blog/ai-solves-imo-problems-at-silver-medal-level/, 2024.